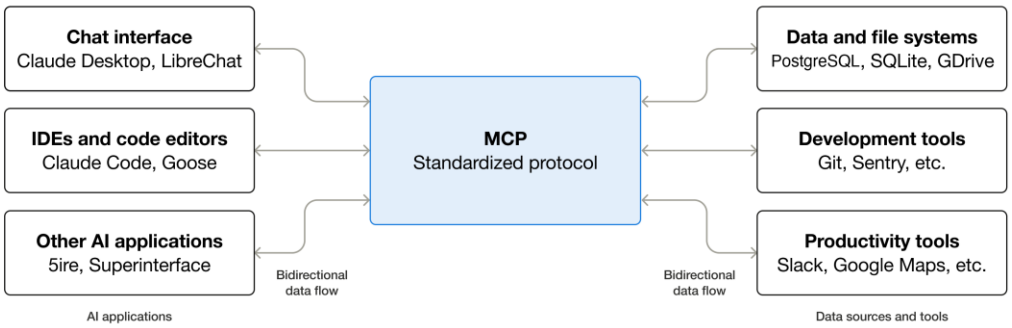
What is MCP?

Model Context Protocol (MCP) is an open protocol that lets AI assistants like Claude securely connect to external tools and data sources. Instead of copying code into chat, Claude can directly read repos, create issues, open PRs, and more — all from inside the terminal.

- **The problem:** Every AI app needed a custom, one-off connector per external system.
  - N apps × M systems → **N×M** bespoke integrations to build, test, secure, version & maintain.
  - Add one tool → re-integrate into every app; add one app → re-integrate every tool.
- **MCP collapses this to N + M**: each app implements the protocol once (host/client), each system once (server).
  - Any compliant client can talk to any compliant server — build once, integrate everywhere.





MCP follows a **client–server architecture**, and there are exactly **three roles** to learn.

1. The **MCP Host** is the AI application itself — for example Claude Desktop or Claude Code — and its job is to coordinate everything.
2. The **MCP Client** is a small component inside the host that maintains a connection to one server and fetches context from it.
3. The **MCP Server** is a program that provides the actual tools, resources, and prompts.

Architecture

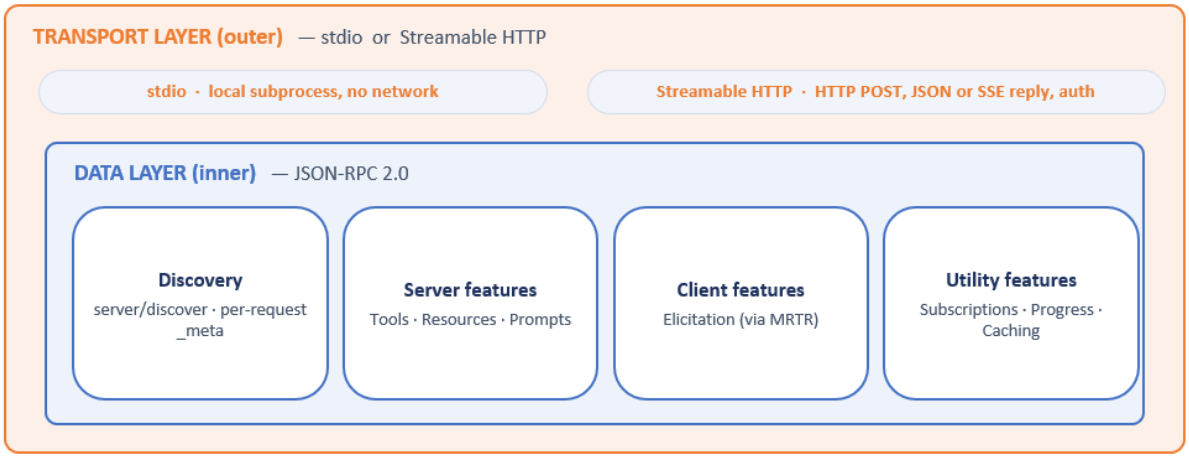
Transport	When to Use
stdio	Local process on your machine (Docker or binary)
Streamable http	Remote cloud-hosted server (Recommended)

1. A local server runs on the same machine as the host, which launches it, and they talk over the STDIO transport — that is simply standard input and output between two programs, with no network

involved. A local server usually serves a single client; a good example is Claude Desktop starting up the filesystem server on your own laptop.

2. A remote server runs somewhere else, on a remote platform, and talks over the Streamable HTTP transport across the network. Because it lives on the network, it can serve many clients at the same time; the official Sentry server is a good example.

The Two Layer Architecture:



Data layer — the inner layer (JSON-RPC 2.0)

- Discovery & metadata — every request carries protocol version + client capabilities in _meta; optional server/discover
- Server features — tools (actions), resources (context), prompts (templates)
- Client features — elicitation: the server asks the user for input via Multi Round-Trip Requests
- Utility features — subscriptions & notifications, progress, caching hints;

Transport layer — the outer layer

- Stdio transport — std in/out between local processes; optimal performance, no network
- Streamable HTTP — HTTP POST to one endpoint; reply as JSON or an SSE stream; bearer tokens / OAuth

– Abstracts communication away from the protocol — same message format on every transport

Server Primitives:

1. **Tools** are executable functions the AI can call to do something — make an API call, run a database query, write a file; the AI discovers them with tools/list and runs them with tools/call.
2. **Resources** are data sources that provide context to read — like file contents or database records; you find them with resources/list and read them with resources/read.
3. **Prompts** are reusable templates that structure how you talk to the model — like a system prompt or a few-shot example; you list them with prompts/list and fetch them with prompts/get.

Claude Code MCP Configuration

In Claude Code, MCP servers can be configured at **local, project, user, or managed scope**. The location determines who can use the server and in which projects.

Scope	Configuration location	Available to
[Local]	~/.claude.json, under the current project's entry	Only you, in the current project
Project	.mcp.json in the project root	Everyone using that project, if the file is shared
User	~/.claude.json, at user scope	Only you, across your projects
Managed	managed-mcp.json in a system directory	Users on machines where an administrator deploys it

File System Local MCP

```
claude mcp add --scope project --transport stdio filesystem -- npx -y @modelcontextprotocol/server-filesystem '.'
```

Other MCP Commands

- claude mcp list - ` shows `Pending approval
- claude mcp get filesystem
- claude mcp remove filesystgem
- claude mcp reset-project-choices resets your approval and rejection choices for MCP servers defined in the current project's **.mcp.json** file.

Note: The /mcp screen helps you inspect connections and authenticate where required.

Example .mcp.json

```
{
  "mcpServers": {
    "filesystem": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "${CLAUDE_PROJECT_DIR}"]
    }
  }
}
```

MCP Authentication

1. Local Servers – No Authentication
2. API KEY / Header: --header "Authorization: Bearer ${KEY}" or headers in .mcp.json
3. OAuth: Add the server, run /mcp, choose **Authenticate**, finish the browser login. Tokens are stored and refreshed

Setup GitHub MCP Server - Option A — Local Server (Docker / Binary)

GitHub MCP Server Overview

GitHub's official MCP server (ghcr.io/github/github-mcp-server) exposes tools across these areas:

- **Repositories** — read files, search code, create/fork repos
- **Issues** — create, list, update, add comments
- **Pull Requests** — create PRs, review diffs, merge
- **Branches** — create, list, delete branches
- **Actions** — trigger workflows, check run status
- **Commits** — get history, compare SHAs

The local server runs as a stdio process on your machine. Claude Code spawns it automatically.

Method 1: Using the CLI (Recommended)

Add the GitHub MCP server locally via Docker

```
claude mcp add --scope project --transport stdio github-local -- docker run -i --rm -e
GITHUB_PERSONAL_ACCESS_TOKEN ghcr.io/github/github-mcp-server
```

MAC: export GITHUB_PERSONAL_ACCESS_TOKEN="your-github-personal-access-token"
Powershell: $env: GITHUB_PERSONAL_ACCESS_TOKEN="your-github-personal-access-token"

Method 2: .mcp.json - JSON Configuration (Manual Edit)

```
{
  "mcpServers": {
    "github": {
      "type": "stdio",
      "command": "docker",
      "args": [
        "run",
        "-i",
        "--rm",
        "-e",
        "GITHUB_PERSONAL_ACCESS_TOKEN",
        "ghcr.io/github/github-mcp-server"
      ],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "ghp_YOUR_TOKEN_HERE"
      }
    }
  }
}
```

OR

```
claude mcp add-json --scope local github-local '{"type":"stdio","command":"docker","args":["run","-i","--rm","-e","GITHUB_PERSONAL_ACCESS_TOKEN","ghcr.io/github/github-mcp-server"]}'
```

Option B — Remote Server (HTTP)

The remote server is hosted by GitHub at https://api.githubcopilot.com/mcp. No Docker needed — Claude Code connects over HTTP using your PAT as a Bearer token.

Add via CLI (One-liner)

GitHub MCP Remote:

```
claude mcp add --scope project --transport http github https://api.githubcopilot.com/mcp/ --header 'Authorization: Bearer ${ GITHUB_PERSONAL_ACCESS_TOKEN }'
```

MAC: export GITHUB_PERSONAL_ACCESS_TOKEN ="your-github-personal-access-token"
Powershell: $env:GITHUB_PERSONAL_ACCESS_TOKEN="your-github-personal-access-token"

Add via JSON (Explicit format)

```
claude mcp add-json github '{
 "type": "http",
 "url": "https://api.githubcopilot.com/mcp",
 "headers": {
  "Authorization": "Bearer GITHUB_PERSONAL_ACCESS_TOKEN"
 }
}'
```

Remote server is great for: quick setup, no Docker, no binary management, insiders/beta features, and teams sharing a common endpoint.

Verifying the Connection

List all configured MCP servers

```
claude mcp list
```

Get details for GitHub specifically

```
claude mcp get github
```

Inside a Claude Code session, check status

```
/mcp
```

You should see github with a checkmark. If it shows pending, wait a moment — Claude Code auto-reconnects with exponential backoff up to 5 attempts.

Importing servers from Claude Desktop

```
claude mcp add-from-claude-desktop
```

Shows the servers in **claude_desktop_config.json** and lets you choose which to copy. Available on **macOS** and **WSL** only.

Useful environment variables

Variable	Purpose
MCP_TIMEOUT	How long (ms) to wait for servers to start, e.g. MCP_TIMEOUT=10000 claude.
MAX_MCP_OUTPUT_TOKENS	Maximum size of a tool result (default 25,000 tokens).
ENABLE_TOOL_SEARCH=false	load every MCP tool up front
MCP_TOOL_TIMEOUT=60000	tool-call timeout, in milliseconds

On Windows PowerShell set them first: $env:MCP_TIMEOUT = "10000"; claude.

Permission rules and environment variables

.CLAUDE/SETTINGS.JSON

```
{
  "permissions": {
    "allow": ["mcp__filesystem__read_text_file", "mcp__filesystem__list_directory"],
    "deny": ["mcp__filesystem__write_file", "mcp__filesystem__edit_file", "mcp__filesystem__move_file",
          "mcp__filesystem__create_directory"]
  }
}
```

Demo Prompts — What You Can Do

Once connected, try these natural language prompts inside Claude Code:

Repository Operations

- List all my public repositories and their star counts.
- Read the README.md from my repo "todo-app" and summarize it.
- Search for files containing "useEffect" in my repo "todo-app".

Issue Management

- Create an issue in my repo "todo-app" titled "Add dark mode support"
- with label "enhancement".
- List all open issues in "todo-app" and group them by label.

Pull Requests

- Create a new branch called "feature/dark-mode" in my "todo-app" repo,
- then open a draft PR titled "WIP: Dark Mode" targeting main.

- Show me the diff for PR #12 in my "todo-app" repo.

End-to-End Development Flow

- Look at issue #5 in my "todo-app" repo, implement the requested feature
- in the current codebase, commit the changes, and open a PR linking to that issue.

Playwright Testing — Todo App Demo

You already have a Todo App in your folder. Here's how to add **Playwright MCP** to Claude Code so Claude can actually open a browser, interact with the app, and validate behavior.

Step 1: Install Playwright MCP Server

Add Playwright MCP to Claude Code

```
claude mcp add --scope user playwright -- npx -y @playwright/mcp@latest
```

Or in JSON format:

```
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": ["-y", "@playwright/mcp@latest"]
    }
  }
}
```

Step 2: Install Playwright in Your Todo App

```
cd todo-app
npm install -D @playwright/test
npx playwright install chromium
```

Step 3: Create a Playwright Test File

Create tests/todo.spec.ts in your Todo App:

```
import { test, expect } from '@playwright/test';
test.describe('Todo App', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('http://localhost:3000');
  });

  test('should load the app and show empty state', async ({ page }) => {
    await expect(page).toHaveTitle(/Todo/i);
    await expect(page.getByText('No todos yet')).toBeVisible();
  });
```

```
test('should add a new todo item', async ({ page }) => {
  const input = page.getByPlaceholder('Add a new todo...');
  await input.fill('Buy groceries');
  await input.press('Enter');

  await expect(page.getByText('Buy groceries')).toBeVisible();
});

test('should mark a todo as complete', async ({ page }) => {
  // Add a todo first
  const input = page.getByPlaceholder('Add a new todo...');
  await input.fill('Complete this task');
  await input.press('Enter');

  // Click the checkbox to complete it
  const checkbox = page.getByRole('checkbox').first();
  await checkbox.check();

  await expect(checkbox).toBeChecked();
});

test('should delete a todo item', async ({ page }) => {
  // Add a todo first
  const input = page.getByPlaceholder('Add a new todo...');
  await input.fill('Delete me');
  await input.press('Enter');

  // Hover to reveal delete button, then click
  const todoItem = page.getByText('Delete me');
  await todoItem.hover();
  await page.getByRole('button', { name: /delete/i }).first().click();

  await expect(page.getByText('Delete me')).not.toBeVisible();
});

test('should filter todos by status', async ({ page }) => {
  // Add two todos
  const input = page.getByPlaceholder('Add a new todo...');
```

```
  await input.fill('Active task');
  await input.press('Enter');
  await input.fill('Completed task');
  await input.press('Enter');

  // Complete the second one
  const checkboxes = page.getByRole('checkbox');
  await checkboxes.nth(1).check();

  // Filter by Active
  await page.getByRole('button', { name: 'Active' }).click();
  await expect(page.getByText('Active task')).toBeVisible();
  await expect(page.getByText('Completed task')).not.toBeVisible();

  // Filter by Completed
  await page.getByRole('button', { name: 'Completed' }).click();
  await expect(page.getByText('Completed task')).toBeVisible();
  await expect(page.getByText('Active task')).not.toBeVisible();
 });

 test('should persist todos after page refresh', async ({ page }) => {
  const input = page.getByPlaceholder('Add a new todo...');
  await input.fill('Persistent task');
  await input.press('Enter');

  await page.reload();

  await expect(page.getByText('Persistent task')).toBeVisible();
 });

});
```

Step 4: Configure playwright.config.ts

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
```

```
  retries: process.env.CI ? 2 : 0,
  reporter: 'html',
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
  },
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
    {
      name: 'Mobile Chrome',
      use: { ...devices['Pixel 5'] },
    },
  ],
  // Automatically start your dev server before running tests
  webServer: {
    command: 'npm run dev',
    url: 'http://localhost:3000',
    reuseExistingServer: !process.env.CI,
  },
});
```

Step 5: Run Tests

```
# Run all tests (headless)
npx playwright test

# Run with UI (watch the browser)
npx playwright test --headed

# Run a specific test file
npx playwright test tests/todo.spec.ts

# Open interactive Playwright UI
npx playwright test --ui

# View the HTML report after a run
```

```
npx playwright show-report
```

Step 6: Let Claude Code Run Playwright via MCP

With Playwright MCP connected, prompt Claude Code like this:

```
Start my Todo app dev server, then use Playwright to:
1. Open the app at localhost:3000
2. Add three todos: "Buy milk", "Go for a run", "Read a book"
3. Mark "Go for a run" as complete
4. Filter by "Active" and take a screenshot
5. Tell me if the filter is working correctly
```

Claude will control the browser in real time, interact with your UI, and report back — no manual testing needed.